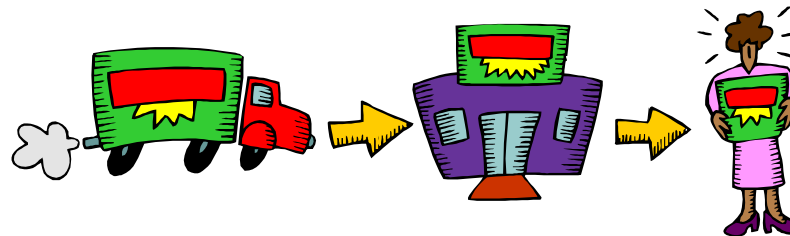


Công nghệ phần mềm

Một số phương pháp kiểm thử cơ bản



Nội dung

- Kiểm thử dựa trên phân tích giá trị biên
- Kiểm thử dựa trên phân hoạch tương đương
- Kiểm thử dựa trên bảng quyết định

Tổng quan về phân tích giá trị biên

- Các chương trình có thể coi là một hàm (toán học)
 - Các đầu vào chương trình là miền xác định của hàm
 - Các đầu ra là miền giá trị của hàm
- Phân tích giá trị biên (boundary value analysis - BVA) là kỹ thuật kiểm thử hàm phổ biến nhất
- Mục tiêu của kiểm thử hàm là sử dụng kiến thức về hàm để xác định các ca kiểm thử
 - Trước kia chủ yếu tập trung vào miền xác định, nhưng nay đã dựa trên cả miền giá trị của hàm để xác định ca kiểm thử

Phân tích giá trị biên (BVA)

- Phân tích giá trị biên tập trung vào biên của miền xác định để xây dựng ca kiểm thử
- Lý do là lỗi thường xảy ra ở gần các giá trị biên này
- Chương trình viết bằng ngôn ngữ không có kiểm tra kiểu mạnh càng cần kiểm thử giá trị biên
 - Javascript, php, Visual Basic

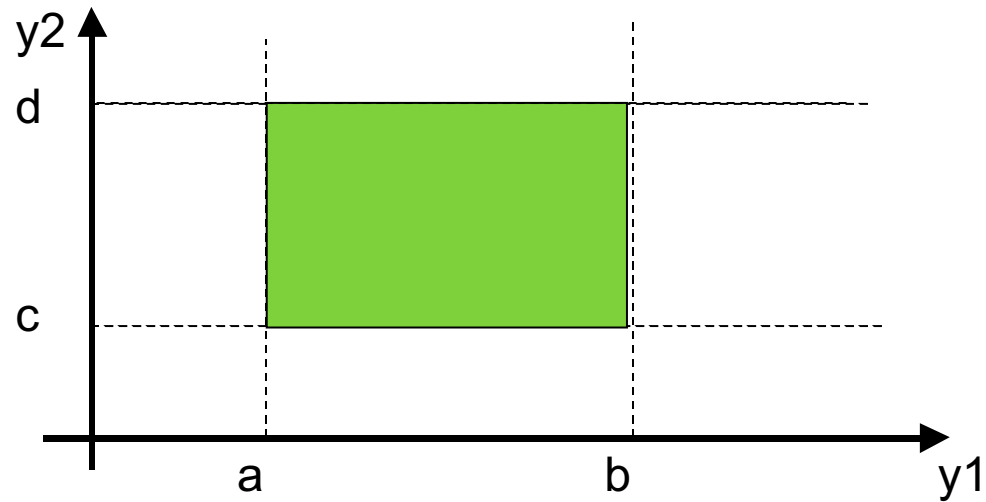
Đầu vào hợp lệ của chương trình P

- Từ đây chúng ta giả sử có chương trình P nhận hai biến đầu vào là y_1 và y_2 thỏa mãn $a \leq y_1 \leq b$ and $c \leq y_2 \leq d$

$P(y_1, y_2)$ where $a \leq y_1 \leq b, c \leq y_2 \leq d$

- Chương trình nhận n đầu vào sẽ có không gian đầu vào n chiều

Miền xác định



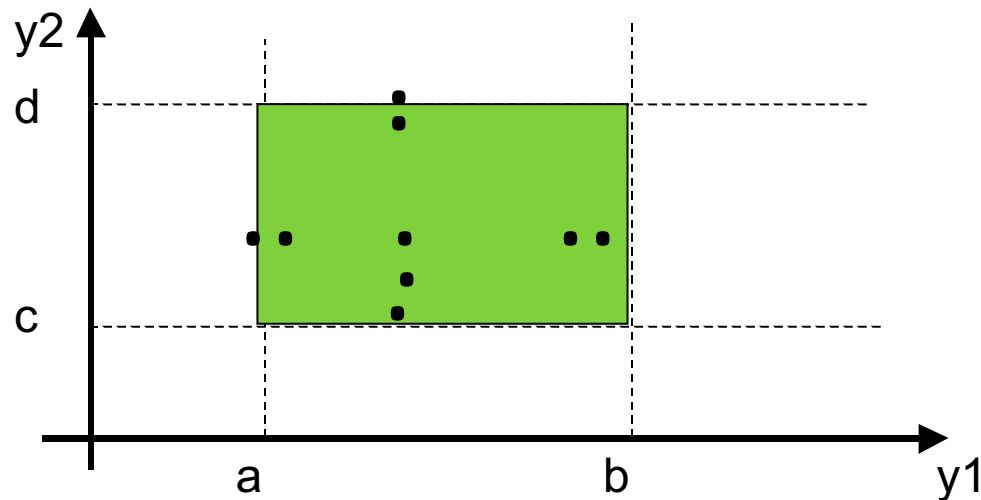
Chọn giá trị

- Phân tích giá trị biên sẽ chọn các giá trị:
 - Giá trị nhỏ nhất
 - Ngay trên giá trị nhỏ nhất
 - Một giá trị bình thường
 - Ngay dưới giá trị lớn nhất
 - Giá trị lớn nhất
- Ví dụ:
 - $a \leq y_1 \leq b$ thì sẽ chọn $a, a+1, a+b/2, b-1, b$.

Giả thiết khiếm khuyết đơn

- Phân tích giá trị biên dựa trên nguyên lý giả định khiếm khuyết đơn:
 - “Hỏng hóc xảy ra hiếm khi do hai (hoặc hơn) khiếm khuyết cùng xảy ra”
- Do đó các ca kiểm thử theo phương pháp này được tạo bằng việc lấy các giá trị bình thường của các chiều/biên rồi lần thay mỗi chiều bằng các giá trị cực trị như trên.

Các ca kiểm thử theo phân tích giá trị biên



$$T = \{ \langle y_{1nom}, y_{2min} \rangle, \langle y_{1nom}, y_{2min+} \rangle, \langle y_{1nom}, y_{2nom} \rangle, \langle y_{1nom}, y_{2max-} \rangle, \langle y_{1nom}, y_{2max+} \rangle, \langle y_{1min}, y_{2nom} \rangle, \langle y_{1min+}, y_{2nom} \rangle, \langle y_{1max-}, y_{2nom} \rangle, \langle y_{1max}, y_{2nom} \rangle \}$$

Các ca kiểm thử cho Triangle

Case #	a	b	c	Expected Output
1	100	100	1	Isosceles
2	100	100	2	Isosceles
3	100	100	100	Equilateral
4	100	100	199	Isosceles
5	100	100	200	Not a Triangle
6	100	1	100	Isosceles
7	100	2	100	Isosceles
8	100	100	100	Equilateral
9	100	199	100	Isosceles
10	100	200	100	Not a Triangle
11	1	100	100	Isosceles
12	2	100	100	Isosceles
13	100	100	100	Equilateral
14	199	100	100	Isosceles
15	200	100	100	Not a Triangle

Tổng quát hóa BVA

- Có hai cách tổng quát hóa :
 - Theo số biến, sẽ có $(4n + 1)$ ca kiểm thử cho n biến
 - Theo loại khoảng của biến
 - Phụ thuộc ngôn ngữ lập trình
 - Tính rời rạc của biến
 - Tính rời rạc không bị chặn (không có cận trên và cận dưới rõ ràng)
 - Biến logic

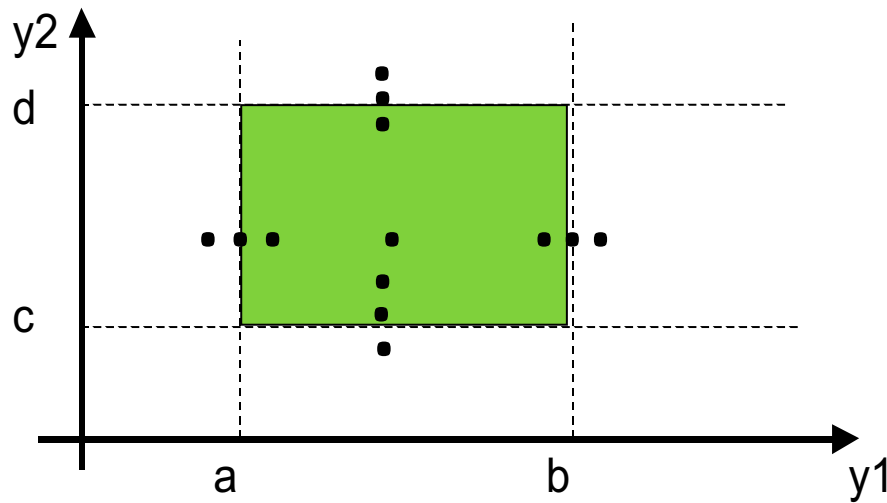
Hạn chế của BVA

- BVA hiệu quả với các chương trình có các đầu vào độc lập nhau và biểu diễn đại lượng vật lý bị chặn
- BVA lấy các ca kiểm thử mà không tính đến chức năng của hàm, hay ý nghĩa của các biến

Kiểm thử biên mạnh

- Kiểm thử biên mạnh (robustness testing) là một mở rộng đơn giản của BVA
- Ngoài năm giá trị biên bổ sung thêm hai giá trị ngoài biên:
 - Giá trị ngay trên giá trị cực đại (max+) và
 - Giá trị ngay dưới giá trị cực tiểu (min-).
- Mục đích chính là xem chương trình có kiểm tra giá trị hợp lệ của đầu vào không.

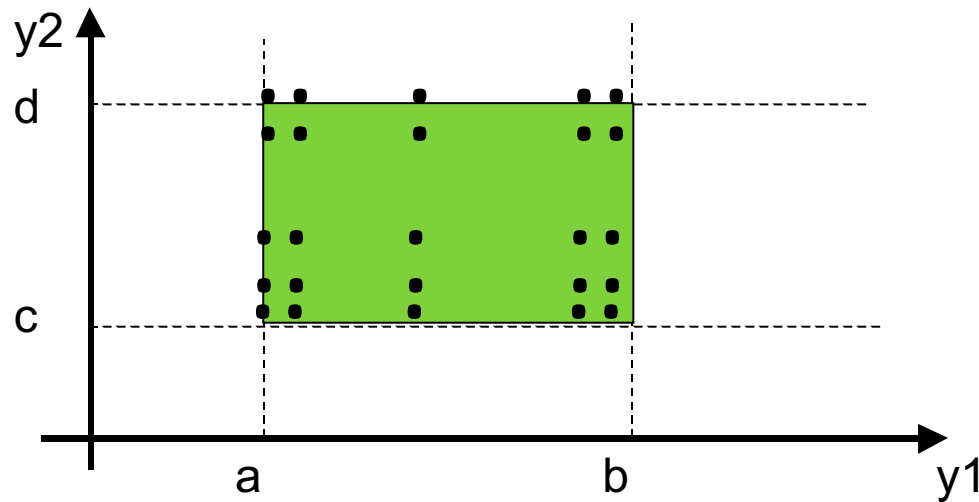
Các ca kiểm thử biên mạnh cho P



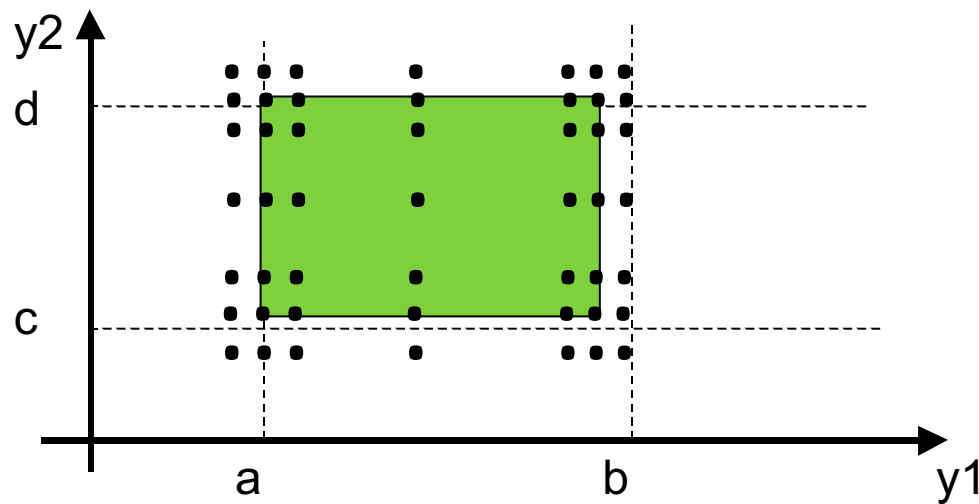
Kiểm thử trường hợp xấu nhất

- Điều gì xảy ra khi nhiều hơn một biến nhận các giá trị (gần) cực trị?
- Khi các biến có tương tác với nhau thì cần kiểm tra các bộ giá trị kết hợp các cực trị này
- Có thể kết hợp với kiểm thử mạnh để có bộ kiểm thử trường hợp xấu nhất mạnh

Các ca kiểm thử trường hợp xấu nhất cho P



Các ca kiểm thử trường hợp xấu nhất mạnh cho P



Kiểm thử giá trị đặc biệt

- Kiểm thử giá trị đặc biệt là phương pháp được thực hiện nhiều nhất trên thực tế, nó cũng trực quan nhất, và không có dạng cố định nhất
- Sử dụng kỹ nghệ và kiến thức miền ứng dụng để phán đoán và đưa ra ca kiểm thử
- Mặc dù mang tính chủ quan cao, đây vẫn là phương pháp hiệu quả để phát hiện khiếm khuyết của chương trình

Kiểm thử lớp tương đương

- Lý do:
 - Cảm giác kiểm thử hết
 - Tránh dư thừa
- Các lớp tương đương tạo thành một phân hoạch của miền dữ liệu
 - Hợp của tất cả các lớp bằng miền đầu vào
 - Cảm giác đã kiểm thử hết
 - Hai lớp bất kỳ không giao nhau
 - Không dư thừa

Các lớp tương đương

- Ý tưởng của ECT là chỉ kiểm thử với một phần tử của mỗi miền tương đương
 - Giảm rất nhiều dư thừa tiềm tàng nếu các lớp tương đương được chọn hợp lý
- Mấu chốt là làm sao chọn được quan hệ tương đương để từ đó xác định được các lớp tương đương (phân hoạch)

Chọn phân hoạch

- Thường là “thủ công” (craft):
 - Không dựa trên mã nguồn, chỉ dựa trên đặc tả
 - Cần hiểu biết về miền xác định, thường không thể xác định dựa vào đặc tả thiết kế giao diện
 - Phải hiểu đầu vào phụ thuộc nhau như thế nào

Ví dụ

- Xét chương trình P có ba biến đầu vào: a, b và c với các miền xác định là A, B, and C.
- Phân hoạch của các miền này giả sử là:

$$A = A1 \cup A2 \cup A3$$

$$B = B1 \cup B2 \cup B3 \cup B4$$

$$C = C1 \cup C2$$

Ví dụ (tiếp)

- Gọi ai thuộc Ai là một phần tử đại diện của lớp
 - Ví dụ lấy phần tử giữa của 1 khoảng
- Tương tự có bi và ci.
- Các ca kiểm thử sẽ được xây dựng từ các phần tử đại diện này
- Ý tưởng ở đây là phần tử đại diện này cũng tốt như các phần tử khác ở trong cùng lớp tương đương đó

ECT yếu

- ECT yếu chỉ lấy tất cả các phần tử đại diện ít nhất một lần
- Số ca kiểm thử tối thiểu sẽ bằng số lớp của phân hoạch có nhiều tập con nhất
 - Trong ví dụ trước là 4

#	a	b	c
WE1	a_1	b_1	c_1
WE2	a_2	b_2	c_2
WE3	a_3	b_3	c_1
WE4	a_1	b_4	c_2

ECT mạnh

- ECT mạnh dựa trên tích Đề-các của các lớp con
- Với ví dụ trước ta có:

$$3 * 4 * 2 = 24 \text{ ca kiểm thử}$$

- Cách này xét đến tất cả các tương tác của các giá trị đại diện

#	a	b	c
SE1	a1	b1	c1
SE2	a1	b1	c2
SE3	a1	b2	c1
SE4	a1	b2	c1
SE5	a1	b3	c1
SE6	a1	b3	c2
SE7	a1	b4	c1
SE8	a1	b4	c2
SE9	a2	b1	c1
SE10	a2	b1	c2
SE11	a2	b2	c1
SE12	a2	b2	c2
SE13	a2	b3	c1
SE14	a2	b3	c2
SE15	a2	b4	c1
SE16	a2	b4	c2
SE17	a3	b1	c1
SE18	a3	b1	c2
SE19	a3	b2	c1
SE20	a3	b2	c2
SE21	a3	b3	c1
SE22	a3	b3	c2
SE23	a3	b4	c1
SE24	a3	b4	c2

ECT truyền thống

- Chỉ phân biệt lớp giá trị hợp lệ và không hợp lệ

Bảng quyết định

- Yêu cầu chức năng có thể mô tả bằng bảng quyết định (DT)
- DT là một cách chính xác và gọn để mô tả logic phức tạp
 - Gắn các điều kiện với các hành động tương ứng
 - Giống lệnh if-then-else và switch-case
- DT có thể liên kết nhiều điều kiện độc lập với vài hành động một cách dễ hiểu
 - Khác các cấu trúc điều khiển trong các ngôn ngữ lập trình

Ví dụ về bảng quyết định

Điều kiện	Máy in không in	Y	Y	Y	Y	N	N	N	N
	Đèn đỏ nhấp nháy	Y	Y	N	N	Y	Y	N	N
	Không nhận ra máy in	Y	N	Y	N	Y	N	Y	N
Hành động	Kiểm tra dây nguồn			X					
	Kiểm tra dây tín hiệu	X		X					
	Kiểm tra phần mềm in đã cài đúng	X		X		X		X	
	Kiểm tra/thay mực	X	X			X	X		
	Kiểm tra kẹt giấy		X		X				

Sử dụng bảng quyết định

- Để quan sát tất cả các điều kiện dễ dàng
- Có thể dùng để
 - Mô tả logic phức tạp
 - Sinh ca kiểm thử, còn gọi là *kiểm thử dựa trên logic*
- Kiểm thử dựa trên logic được xem là:
 - Kiểm thử cấu trúc khi áp dụng cho các cấu trúc chương trình
 - Vd luồng điều khiển
 - Kiểm thử hàm khi áp dụng cho đặc tả.

Cấu trúc bảng quyết định

1. Các điều kiện	2. Các giá trị điều kiện
3. Hành động	4. Xảy ra hay không

1. Mỗi điều kiện tương ứng với một biến, một quan hệ, hoặc một mệnh đề (predicate)
2. Các giá trị của điều kiện
 - Chỉ là True/False – Bảng quyết định hạn chế
 - Một số giá trị – Bảng quyết định mở rộng
 - Giá trị không quan tâm
3. Mỗi hành động là một thủ tục hoặc thao tác phải thực hiện
4. Đánh dấu hành động có/không xảy ra

Ví dụ bảng quyết định tính lương

	Conditions/ Courses of Action	Rules					
		1	2	3	4	5	6
Condition Stubs	Employee type	S	H	S	H	S	H
	Hours worked	<40	<40	40	40	>40	>40
Action Stubs	Pay base salary	X		X		X	
	Calculate hourly wage		X		X		X
	Calculate overtime						X
	Produce Absence Report		X				

Cách tính lương

Phương pháp xây dựng bảng

1. Xác định các điều kiện và giá trị của chúng
2. Xác định số luật tối đa
3. Xác định các hành động
4. Đánh số các luật nếu cần
5. Đánh số các hành động thích hợp cho mỗi luật
6. Kiểm tra chính sách
7. Đơn giản hóa các luật (gộp cột)

Sử dụng bảng quyết định

- Bảng thích hợp khi:
 - Đặc tả có thể chuyển về dạng bảng
 - Thứ tự các hành động xảy ra không quan trọng
 - Thứ tự các luật không ảnh hưởng đến hành động
 - Khi một luật thỏa mãn và được chọn thì không cần xét luật khác
- Các hạn chế trên không ảnh hưởng đến việc sử dụng bảng
 - Trong hầu hết các ứng dụng thứ tự các mệnh đề được xét là không quan trọng

Một số vấn đề với bảng quyết định

- Trước khi sử dụng bảng cần đảm bảo:
 - Các luật phải đầy đủ
 - Có mọi tổ hợp
 - Các luật phải nhất quán
 - Mọi tổ hợp giá trị chân lý chỉ gây ra một hoặc một tập hành động

Thiết kế ca kiểm thử

- Khi đặc tả đã được kiểm tra, mục tiêu là chứng tỏ chương trình thực hiện các hành động đúng cho mọi tổ hợp các giá trị của mệnh đề
 - Nếu có k luật và n mệnh đề đúng/sai, thì có ít nhất k trường hợp và nhiều nhất là 2^n trường hợp phải xét.
- Có thể dựa trên các luật chưa mở rộng hoặc các luật đã mở rộng với 2^n ca
 - Xác định đầu vào cho mỗi ca

Thiết kế ca kiểm thử

- Để xác định ca kiểm thử, chúng ta chuyển các điều kiện thành đầu vào, hành động thành đầu ra.
- Một số điều kiện sẽ tham chiếu đến các lớp tương đương đầu vào, và hành động tham chiếu đến các phần xử lý chức năng chính của cột đang xét.
- Các luật được chuyển thành các ca kiểm thử

Tổng kết

- Kiểm thử dựa trên phân tích giá trị biên
- Kiểm thử dựa trên phân hoạch tương đương
- Kiểm thử dựa trên bảng quyết định